author: Harzarul

> bootcamp.learn()

JOMHACK
EMPOWERING INNOVATION

JomCode Coding Bootcamp

CHAPTER 22

TypeScript for application code

learn / type / run / explain

Plain meaning and technical view

PLAIN MEANING

TypeScript checks the shapes your application expects before the browser runs it.

TECHNICAL VIEW

Interfaces, unions, narrowing, unknown and small generic helpers describe values. Types disappear at runtime.

REAL-WORLD USE

Keep Postlight API records, form state and request helpers consistent.

QUICK TIP

Use unknown for external data and narrow it. A type assertion cannot validate server JSON.

INSTALL / RUN

In typescript-practice: install TypeScript/tsx once, then npm run check:ts and npm run demo:ts. Setup commands and config follow.

Why this chapter uses npm

DO THIS / WHY

npm adds a consistent TypeScript toolchain: tsc checks types; tsx runs TS. Running code alone, including Node's built-in TS support, is not type checking. package.json declares tools/scripts; package-lock.json records exact versions; node_modules contains packages. Commit both JSON files, not node_modules.

CHECKPOINT / TIP

This is supporting setup; the four worked examples follow next.

Create typescript-practice / once

DO THIS / WHY

Create a sibling practice folder, not inside frontend-practice. -D marks development tools. Install once for a fresh folder; rerun when dependencies change. npm ci is for an existing lockfile.

CHECKPOINT / TIP

Use the named folder and your OS command. Keep long-running servers in separate terminals.

Create typescript-practice / once / terminal

terminal

```
> mkdir typescript-practice

> cd typescript-practice

> npm init -y

> npm install -D typescript@~5.9.3 tsx@^4.20.0
```

Type commands only. Omit the > prompt and OS labels.

Edit package.json / keep other fields

DO THIS / WHY

Set type to module. Replace scripts with the two entries below. Preserve the devDependencies npm just added.

CHECKPOINT / TIP

JSON needs double quotes and no trailing comma after its final field.

Edit package.json / keep other fields

package.json (fields, not a complete file)

+ = same line

```
01   "type": "module",
02   "scripts": {
03     "check:ts": "tsc --noEmit",
04     "demo:ts": "tsx chapter_22_typescript_for_application_code.ts"
05   }
```

PRO TIP JSON needs double quotes and no trailing comma after its final field.

Type-checker configuration

DO THIS / WHY

Create tsconfig.json. Save the chapter TS file with shared setup at the top. Wrap each example and exercise in its own { ... } block to isolate repeated type/variable names.

CHECKPOINT / TIP

strict checks more mistakes. noEmit checks without generating JavaScript files.

Type-checker configuration (1/2)

tsconfig.json

+ = same line

```
01  {
02    "compilerOptions": {
03      "target": "ES2022",
04      "module": "ESNext",
05      "moduleResolution": "Bundler",
06      "lib": [
07        "ES2022",
08        "DOM"
09      ],
```

PRO TIP strict checks more mistakes. noEmit checks without generating JavaScript files.

Type-checker configuration (2/2)

tsconfig.json

+ = same line

```
10        "strict": true,
11        "noEmit": true,
12        "skipLibCheck": true
13      },
14      "include": [
15        "*.ts"
16      ]
17    }
```

PRO TIP strict checks more mistakes. noEmit checks without generating JavaScript files.

Check first, then run

DO THIS / WHY

Run in typescript-practice. check:ts should exit without errors. demo:ts prints No cover, Pytho, Hello and Hello for the four examples. tsx execution alone does not prove the types passed.

CHECKPOINT / TIP

Use the named folder and your OS command. Keep long-running servers in separate terminals.

Check first, then run / terminal

terminal

```
> npm run check:ts

> npm run demo:ts
```

Type commands only. Omit the > prompt and OS labels.

Shared setup

chapter_22_typescript_for_application_code.ts

+ = same line

```ts
export {}; // A module permits top-level await.
```

PRO TIP Keep these imports, definitions and fixtures when running the examples.

Worked example 1: Post shape and nullability

chapter_22_typescript_for_application_code.ts

+ = same line

```
01  type Post = { id: string; title: string; image_url: string | null };
02  const post: Post = { id: "p1", title: "Hello", image_url: null };
03  console.log(post.image_url ?? "No cover");
```

PRO TIP Nullable means present with null; optional means it may be absent.

Run and explain example 1

EXPECTED OUTPUT

No cover

WHY IT BEHAVES THIS WAY

Yes. A nullable field is present but may be null. An optional field may be absent.

Worked example 2: Typed function and narrowing

chapter_22_typescript_for_application_code.ts

+ = same line

```
01  function preview(title: string, length = 5): string {
02    return title.slice(0, length);
03  }
04  let selected: { title: string } | null = { title: "Python" };
05  if (selected !== null) console.log(preview(selected.title));
```

PRO TIP Narrow a union before accessing fields that may not exist.

Run and explain example 2

EXPECTED OUTPUT

Pytho

WHY IT BEHAVES THIS WAY

The union includes null. Narrowing proves which member is available in that branch.

Worked example 3: Validate unknown data

chapter_22_typescript_for_application_code.ts

+ = same line

```ts
type RecordData = { title: string };
function isRecord(value: unknown): value is RecordData {
  return (
    typeof value === "object" &&
    value !== null &&
    "title" in value &&
    typeof value.title === "string"
  );
}
const incoming: unknown = JSON.parse('{"title":"Hello"}');
if (isRecord(incoming)) console.log(incoming.title);
```

PRO TIP unknown plus a guard checks runtime data; an assertion does not.

Run and explain example 3

EXPECTED OUTPUT

Hello

WHY IT BEHAVES THIS WAY

The guard performs a runtime check. An assertion only tells the compiler to trust you.

Worked example 4: A typed request helper

chapter_22_typescript_for_application_code.ts

+ = same line

```
async function request<T>(url: string): Promise<T> {
  const response = await fetch(url);
  if (!response.ok) throw new Error("HTTP " + response.status);
  return response.json() as Promise<T>;
}
const url = 'data:application/json,{"title":"Hello"}';
const result = await request<{ title: string }>(url);
console.log(result.title);
```

PRO TIP A generic describes the expected result, not a runtime validation.

Run and explain example 4

EXPECTED OUTPUT

Hello. The data URL makes this demo runnable without a server.

WHY IT BEHAVES THIS WAY

No. T connects caller expectations to the return type. Use a runtime guard like example 3 when trusting external data.

Exercise 1: Post contract

YOUR TASK

Define Post and a title preview function. Accept nullable image_url. Narrow selected before reading it.

CHECK THESE CASES

Valid object compiles. Preview returns Pytho. null image uses fallback.

HINT AFTER A TRY

Run the compiler as well as the file.

Exercise 2: Repair unsafe external data

YOUR TASK

Replace any with unknown and validate id/title strings before using the response. Check a malformed object.

CHECK THESE CASES

Good shape passes. Numeric title fails. Code passes strict tsc.

HINT AFTER A TRY

Compile-time types cannot sanitize network data.

Checkpoint before moving on

SHOW AND EXPLAIN

Run and explain all four examples. Complete both exercises and verify every listed acceptance case.

TRANSFER TO POSTLIGHT

Keep Postlight API records, form state and request helpers consistent.